

# WEEK 1,2 JAVA LAB PROGRAM

## Bank Management System Using Inheritance

### 1. Steps

1. Create a base class named account.
2. Declare account properties such as account number, balance, and account type.
3. Create a constructor in the account class to initialize these properties.
4. Define methods for:
  - Deposit
  - Withdrawal
  - Interest calculation
  - Money transfer
  - Displaying account details
5. Create a savingaccount class that inherits from the account class.
6. Add an interest property to the savingaccount class.
7. Override the calinterest() method to calculate interest for a savings account.
8. Create a currentaccount class that inherits from the account class.
9. Add an overdraft facility to the currentaccount class.
10. Override the withdraw() method to allow withdrawal using the overdraft limit.
11. Create objects s1 and c1 for savings and current accounts.
12. Perform deposit, withdrawal, interest calculation, and money transfer operations.
13. Display the final account details.

---

### 2. Algorithm

#### Step 1: Start

Start the program.

## Step 2: Create Account Class

Create the account class with the following data members:

- accno
- balance
- acctype

Initialize them using a constructor.

## Step 3: Define Account Operations

Define the following methods:

- deposit() → Adds money to the account balance.
- withdraw() → Checks whether sufficient balance is available and withdraws money.
- calinterest() → Used for calculating interest.
- transfer() → Transfers money from one account to another.
- display() → Displays account details.

## Step 4: Create Savings Account

Create savingaccount by extending account.

Add the interest variable and override calinterest() to calculate interest based on the balance.

## Step 5: Create Current Account

Create currentaccount by extending account.

Add an overdraft variable and override withdraw() so that the customer can withdraw money up to the overdraft limit.

## Step 6: Create Objects

Create:

- Savings account s1 with account number 101, balance 20000, and interest 5%.
- Current account c1 with account number 102, balance 15000, and overdraft limit 5000.

## Step 7: Perform Operations

For the savings account:

1. Display details.
2. Deposit 5000.
3. Withdraw 3000.
4. Calculate interest.

For the current account:

1. Display details.
2. Deposit 2000.
3. Withdraw 18000.

### Step 8: Transfer Money

Transfer 4000 from the savings account to the current account.

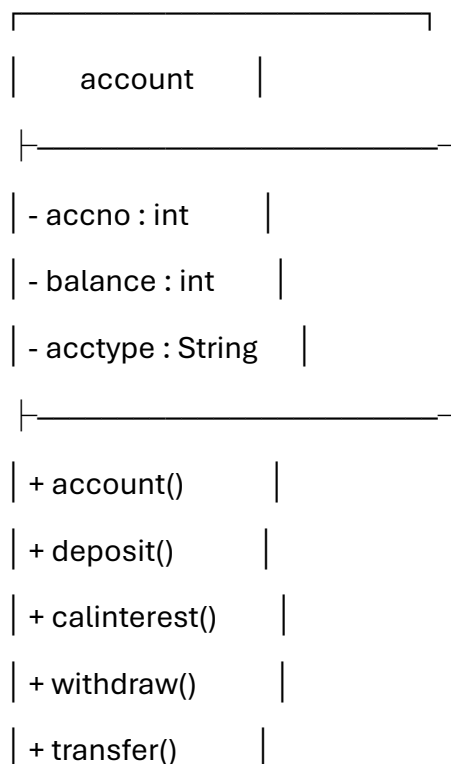
### Step 9: Display Final Details

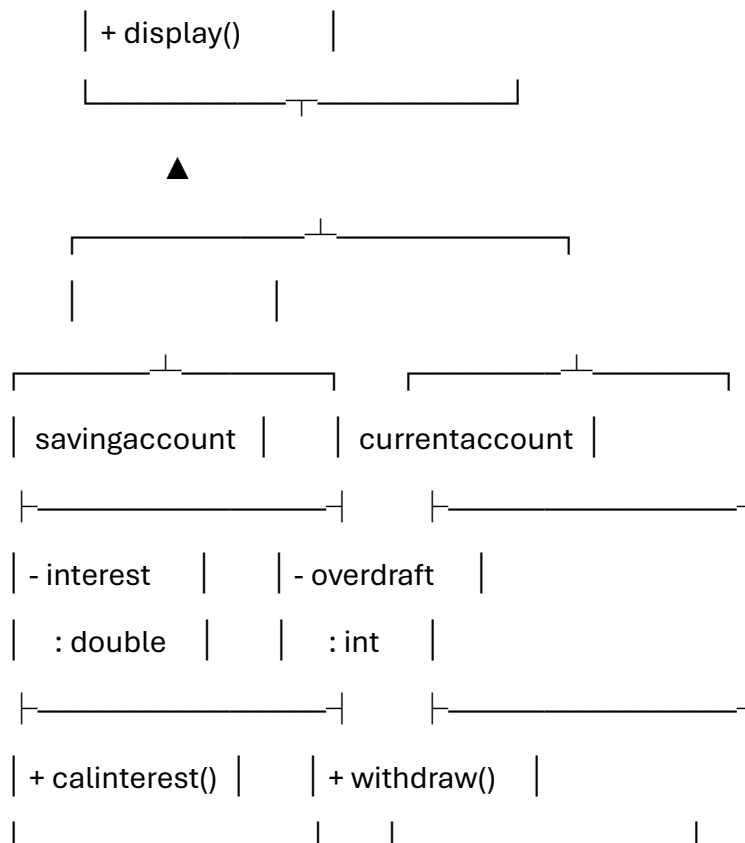
Display the final balance and account details of both accounts.

### Step 10: Stop

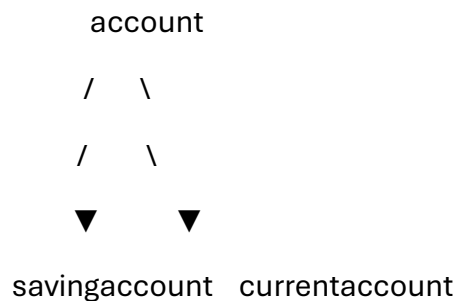
End the program.

## 3. UML Diagram





### Inheritance Relationship



- savingaccount extends account
- currentaccount extends account

This is an example of **hierarchical inheritance**, because two child classes inherit from the same parent class.

---

### 4. Code

```

class account{
    int accno;
    int balance;
  
```

```
String acctype;
```

```
public account(int accno,int balance,String acctype){  
    this.accno=accno;  
    this.balance=balance;  
    this.acctype=acctype;  
}
```

```
void deposit(int amount){  
    balance=balance+amount;  
    System.out.println("balance after deposit is:"+balance);  
}
```

```
void calinterest(){  
}
```

```
void withdraw(int amount){  
    if (balance>=amount) {  
        balance=balance-amount;  
        System.out.println("balance after deposit is :"+balance);  
    }  
    else  
        System.out.println("insufficient balance");  
}
```

```
void transfer(account recevier,int amount){  
    if (balance>=amount) {  
        balance=balance-amount;
```

```
recevier.balance=recevier.balance+amount;
```

```
System.out.println("balance after transfer is :"+recevier.balance);
```

```
}
```

```
else
```

```
System.out.println("insufficent balance");
```

```
}
```

```
void display(){
```

```
System.out.println(accno);
```

```
System.out.println(acctype);
```

```
System.out.println(balance);
```

```
}
```

```
}
```

```
class savingaccount extends account{
```

```
double interest;
```

```
public savingaccount(int accno,int balance,double interest){
```

```
super(accno, balance, "saving");
```

```
this.interest=interest;
```

```
}
```

```
@Override
```

```
void calinterest(){
```

```
interest=interest*balance/100;
```

```
System.out.println("interest is:"+interest);
```

```
}
```

```
}
```

```
class currentaccount extends account{  
    int overdraft;  
  
    public currentaccount(int accno,int balance, int overdraft){  
        this.overdraft=overdraft;  
        super(accno, balance,"current account");  
    }  
}
```

```
    public void withdraw(double amount) {  
        if (amount <= balance + overdraft) {  
            balance -= amount;  
            System.out.println("Amount Withdrawn: " + amount);  
        } else {  
            System.out.println("Overdraft Limit Exceeded");  
        }  
    }  
}
```

```
public class BankManagement {  
    public static void main(String[] args) {  
  
        savingaccount s1 = new savingaccount(101, 20000, 5);  
        currentaccount c1 = new currentaccount(102, 15000, 5000);  
  
        System.out.println("Savings Account");  
        s1.display();  
    }  
}
```

```
s1.deposit(5000);
```

```
s1.withdraw(3000);
```

```
s1.calinterest();
```

```
System.out.println("\nCurrent Account");
```

```
c1.display();
```

```
c1.deposit(2000);
```

```
c1.withdraw(18000);
```

```
System.out.println("\nTransfer Money");
```

```
s1.transfer(c1, 4000);
```

```
System.out.println("\nFinal Details");
```

```
s1.display();
```

```
c1.display();
```

```
}
```

```
}
```

```
}
```

```
PS C:\Users\navee\OneDrive\Desktop\javaprograms\labprograms\Week 1,2> java BankManagement
101
savings
20000
balance after deposit is:25000
balance after deposit is :22000
interest is:1100.0

Current Account
102
current account
15000
balance after deposit is:17000
insufficient balance

Transfer Money
balance after transfer is :21000

Final Details
101
savings
18000
102
current account
21000
```

```
}
```